

AI INVENTORY OPTIMIZER

DATA REPORT

Milagros Pumasupa

Ahad Maredia

Larry Towett

Yasmin Bello

ITAI 2277: Artificial Intelligence Resource

Professor: Anna Devarakonda

October, 2025

AI INVENTORY OPTIMIZER

1. PROBLEM DEFINITION

Stores face ongoing challenges in efficiently managing their inventory. Overstocking results in tied-up capital and losses due to expired products, while understocking high-demand items leads to lost sales and decreases customer satisfaction. Additionally, these stores struggle to quickly identify emerging products and market trends that could boost their sales.

This project aims to develop an AI-powered system that predicts weekly product demand, recommends new or trending products, and provides an interactive dashboard with actionable insights. The goal is to optimize inventory management, reduce waste, maximize product availability, and ultimately increase business profitability.

By adopting this AI-powered system, store owners can make smarter, data-driven decisions that directly impact their bottom line. Instead of relying on intuition or manual tracking, they gain access to precise demand forecasts and timely product recommendations. This means fewer stockouts, less waste from unsold inventory, and improved customer satisfaction. Ultimately, the system not only streamlines operations but also empowers business owners to stay ahead of market trends and outperform competitors.

2. DATA SOURCES AND ACQUISITION

For the development of this project, data published on Kaggle was selected. This dataset contains weekly sales, by store, by department, financial information, and identifies holidays; these variables are important elements for predictions. This data was acquired manually from Kaggle, and code could later be added to automatically pull real data from the databases of stores that want to use the AI Inventory Optimizer.

```
uploaded = files.upload()
```

We confirm that the dataset does not contain any personal or sensitive information, as it is fully anonymized and aggregated at the store level. However, we remain aware of the potential for indirect biases in model predictions. For example, stores located in economically disadvantaged areas or smaller store formats might experience consistent underprediction. As part of our

validation process, we plan to monitor and mitigate such biases to ensure fair and equitable model performance across different store types and regions.

Source: Kaggle - Walmart Recruiting: Store Sales Forecasting

Files used:

- **train.csv:** Historical weekly sales data per store and department
- **features.csv:** External variables (e.g., fuel prices, markdown events, CPI, unemployment)
- **stores.csv:** Metadata describing each store (e.g., type, size)

Train data:

	Store	Dept	Date	Weekly_Sales	IsHoliday
0	1	1	2010-02-05	24924.50	False
1	1	1	2010-02-12	46039.49	True
2	1	1	2010-02-19	41595.55	False
3	1	1	2010-02-26	19403.54	False
4	1	1	2010-03-05	21827.90	False

Features data:

	Store	Date	Temperature	Fuel_Price	Markdown1	Markdown2	\
0	1	2010-02-05	42.31	2.572	NaN	NaN	
1	1	2010-02-12	38.51	2.548	NaN	NaN	
2	1	2010-02-19	39.93	2.514	NaN	NaN	
3	1	2010-02-26	46.63	2.561	NaN	NaN	
4	1	2010-03-05	46.50	2.625	NaN	NaN	

	Markdown3	Markdown4	Markdown5	CPI	Unemployment	IsHoliday
0	NaN	NaN	NaN	211.096358	8.106	False
1	NaN	NaN	NaN	211.242170	8.106	True
2	NaN	NaN	NaN	211.289143	8.106	False
3	NaN	NaN	NaN	211.319643	8.106	False
4	NaN	NaN	NaN	211.350143	8.106	False

Stores data:

	Store	Type	Size
0	1	A	151315
1	2	A	202307
2	3	B	37392
3	4	A	205863
4	5	B	34875

These files were temporarily stored in Google Colab's storage and read with panda. The data was then reviewed and the structure analyzed to finally merge them into a single data frame called: **walmart_preprocessed.csv**.

```
train_df = pd.read_csv('train.csv')
features_df = pd.read_csv('features.csv')
stores_df = pd.read_csv('stores.csv')

#Final preprocessed dataset

merged.to_csv('walmart_preprocessed.csv', index=False)
```

3. DATA PREPROCESSING AND FEATURE ENGINEERING

The file **walmart_preprocessed.csv** represents the final, cleaned, and feature-engineered dataset used for machine learning model development. It was created by merging three data sources:

- **train.csv**: Historical weekly sales data per store and department.
- **features.csv**: External variables such as fuel prices, markdown events, CPI, and unemployment.
- **stores.csv**: Store metadata including store type and size.

Several preprocessing techniques were applied to prepare the data for model training.

3.1 Handling Missing Values

To address missing data:

- All missing values in the MarkDown1 to MarkDown5 columns were filled with zeros, assuming that the absence of data meant no promotional activity occurred.
- Other continuous variables such as CPI, Unemployment, Fuel_Price, and Temperature were forward-filled using the last available value to maintain time continuity.

Code Example:

```
# Fill missing values in Markdown columns with 0 (assume no promotion)
for col in ['MarkDown1', 'MarkDown2', 'MarkDown3', 'MarkDown4', 'MarkDown5']:
    if col in features_df.columns:
        features_df[col] = features_df[col].fillna(0)

# Fill other missing numerical values (CPI, Unemployment, etc.) using forward fill
features_df[['CPI', 'Unemployment', 'Fuel_Price', 'Temperature']] = features_df[['CPI', 'Unemployment', 'Fuel_Price', 'Temperature']].fillna(method='ffill')
```

3.2 Merging Datasets

The three datasets were merged to create a single, comprehensive dataset.

- First, train.csv and features.csv were merged using the Store, Date, and IsHoliday columns.

- Then, the resulting dataset was joined with stores.csv using the Store identifier.

Code Example:

```
# Merge train and features on Store, Date, IsHoliday
merged = pd.merge(train_df, features_df, on=['Store', 'Date', 'IsHoliday'], how='left')

# Merge with store information
merged = pd.merge(merged, stores_df, on='Store', how='left')
```

3.3 Feature Engineering

To enrich the dataset and improve model performance, multiple new features were created.

3.3.1 Date-Based Features

New features were extracted from the Date column to capture time-based trends:

- Year, Month, and Week to detect seasonality
- DayOfWeek to capture shopping patterns across weekdays
- IsWeekend as a binary variable to highlight weekend effects

Code Example:

```
# Extract year, month, week from the date
merged['Year'] = merged['Date'].dt.year
merged['Month'] = merged['Date'].dt.month
merged['Week'] = merged['Date'].dt.isocalendar().week
merged['DayOfWeek'] = merged['Date'].dt.dayofweek # Monday=0, Sunday=6
merged['IsWeekend'] = merged['DayOfWeek'].isin([5, 6]).astype(int)
```

3.3.2 Promotion Aggregation

A new variable Total_MarkDown was created by summing all markdown columns to represent the total promotional activity in a given week.

Code Example:

```
merged['Total_MarkDown'] = (merged['MarkDown1'] + merged['MarkDown2'] +
                           merged['MarkDown3'] + merged['MarkDown4'] +
                           merged['MarkDown5'])
```

3.4 Encoding Categorical Variables

Categorical variables were transformed into numerical format for model compatibility:

- The Type column (A, B, C) and the IsHoliday column were encoded using **one-hot encoding**.

Code Example:

```
# One-hot encoding
merged = pd.get_dummies(merged, columns=['Type', 'IsHoliday'], drop_first=True)
```

3.5 Normalizing Numerical Features

To ensure that variables like Fuel_Price, Temperature, CPI, and Unemployment have comparable scales, Min-Max Scaling was applied.

Code Example:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
merged[['CPI', 'Unemployment', 'Fuel_Price', 'Temperature']] = scaler.fit_transform(
    merged[['CPI', 'Unemployment', 'Fuel_Price', 'Temperature']]
)
```

3.6 Defining Model Features and Train-Test Split

A set of relevant features was selected to train the model. The dataset was split into training and testing sets using an 80-20 ratio while maintaining the chronological order.

Code Example:

```
features = ['Store', 'Dept', 'Size', 'Temperature', 'Fuel_Price', 'CPI', 'Unemployment',
            'Total_MarkDown', 'Month', 'Week', 'DayOfWeek', 'IsWeekend',
            'Type_B', 'Type_C', 'IsHoliday_True'] # Include one-hot columns

X = merged[features]
y = merged['Weekly_Sales']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

3.7 Final Output

Upon completion of the data preprocessing and feature engineering stages, the final dataset was compiled, incorporating all relevant and engineered features necessary for predictive modeling. This fully cleaned and structured dataset was exported as **walmart_preprocessed.csv**.

Additionally, the feature matrix (X) and the target variable (Weekly_Sales) were extracted and saved separately for use in training and evaluating machine learning models. The data was split into training and test sets to maintain the temporal integrity of the series and to facilitate robust model validation.

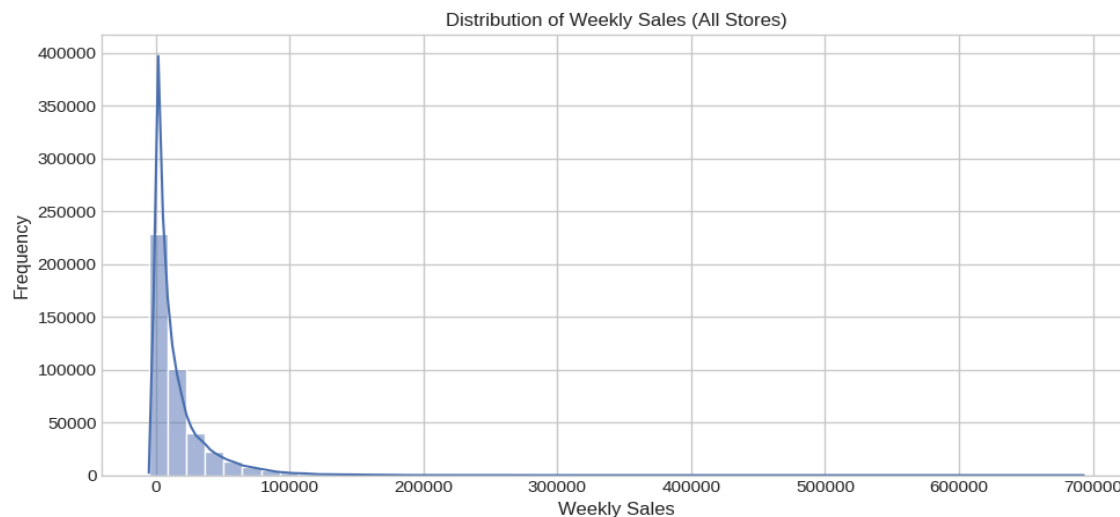
For a complete description of each feature, refer to Appendix 1: Feature Dictionary.

4. EXPLORATORY DATA ANALYSIS (EDA)

After preparing the dataset, we performed Exploratory Data Analysis (EDA) to uncover patterns, trends, and relationships that influence weekly sales. Using a combination of histograms, line plots, bar charts, and boxplots, we examined the distribution of sales, seasonal effects, holiday impacts, and store-level performance. The following visualizations highlight the most relevant insights drawn from the data.

4.1 Distribution of Weekly Sales

This plot shows the overall distribution of the target variable, Weekly_Sales, across all stores and departments.



Insight:

The sales distribution is heavily right-skewed, indicating that while most department-week combinations have moderate sales, a few experience extremely high sales, likely driven by large stores or seasonal peaks.

4.2 Average Weekly Sales by Store Type

This chart compares the average sales performance across store types A, B, and C.

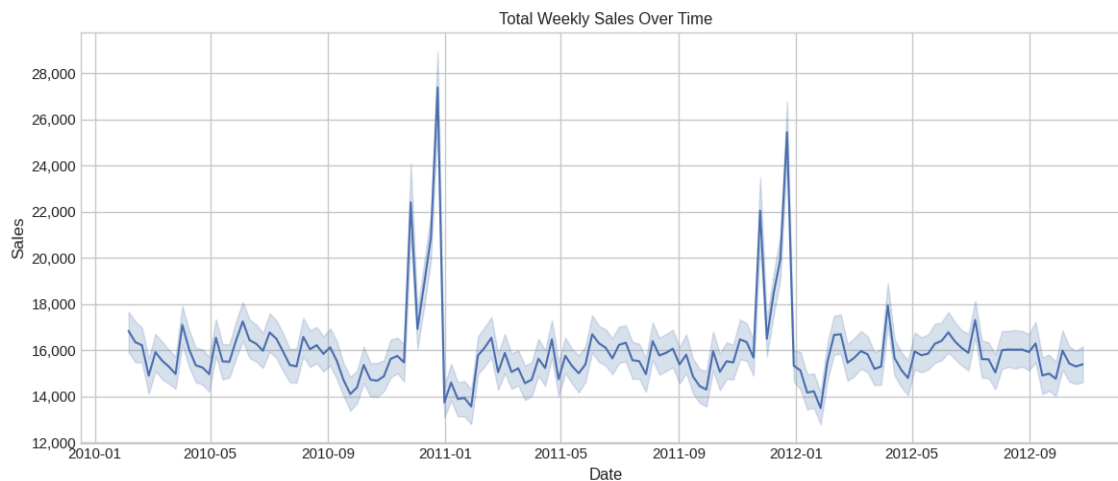


Insight:

Type A stores (usually larger) generate higher average weekly sales than Types B and C, confirming that store format and scale influence revenue.

4.3 Weekly Sales Over Time

This time series shows total sales aggregated across all stores per week.

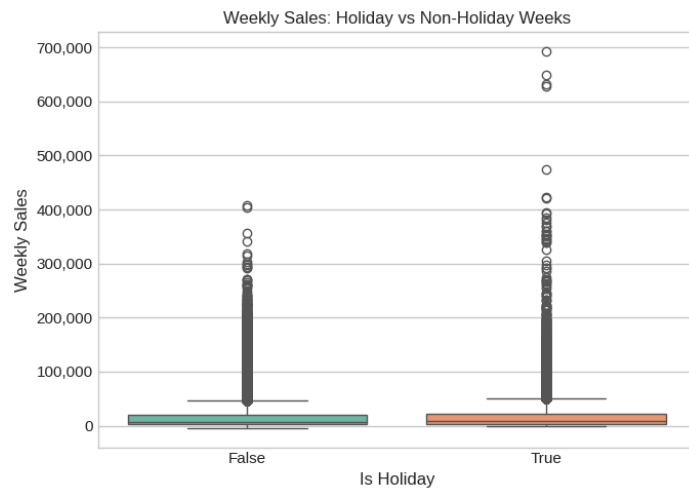


Insight:

Distinct sales peaks are observed around major holidays such as Thanksgiving and Christmas, emphasizing the importance of accounting for holiday seasonality in demand forecasting.

4.4 Weekly Sales During Holiday vs Non-Holiday Weeks

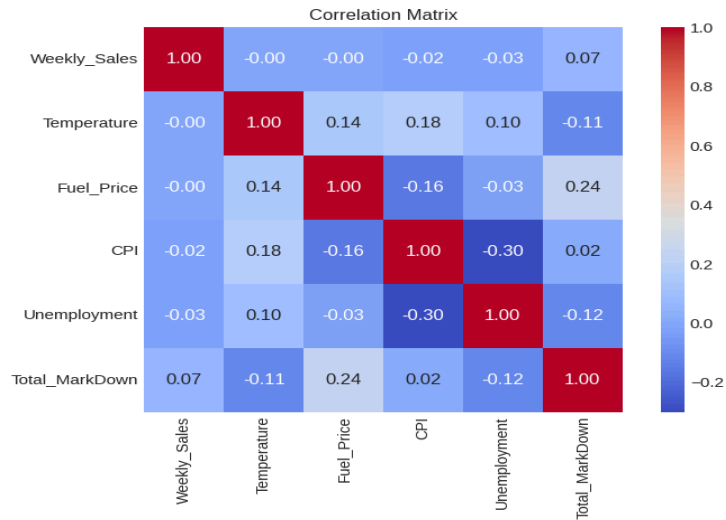
This boxplot contrasts the distribution of sales between holiday and regular weeks.

**Insight:**

Sales are generally higher during holiday weeks, although variability also increases, suggesting that promotions and seasonal demand play a major role during these periods.

4.5 Correlation Heatmap

This heatmap shows the linear correlations between sales and key continuous variables.

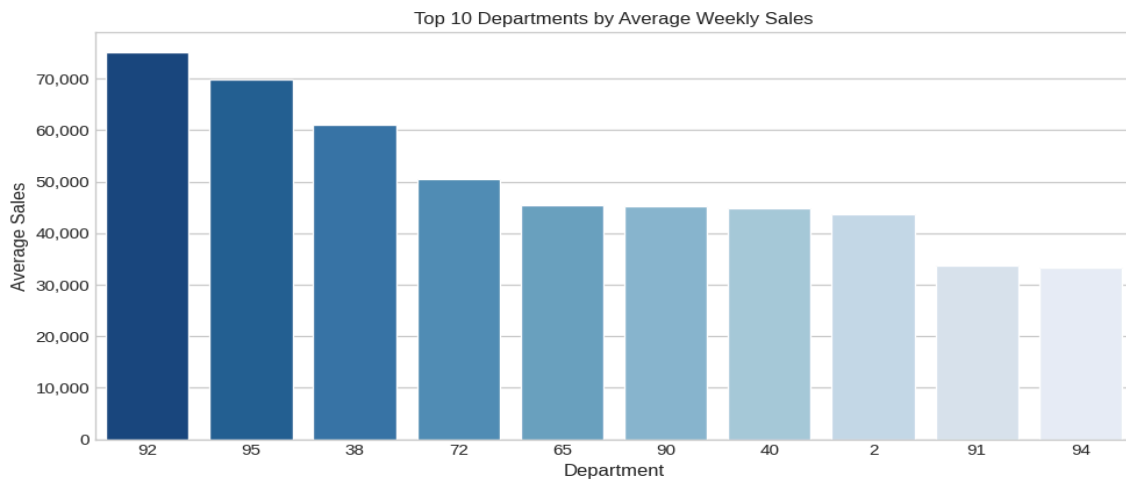


Insight:

Most external variables show weak correlations with weekly sales, except for Total_MarkDown, which exhibits a mild positive correlation, indicating the potential impact of promotional discounts.

4.6 Top 10 Stores by Weekly Sales

Identifying the highest-performing stores provides insight into location-specific sales behavior.

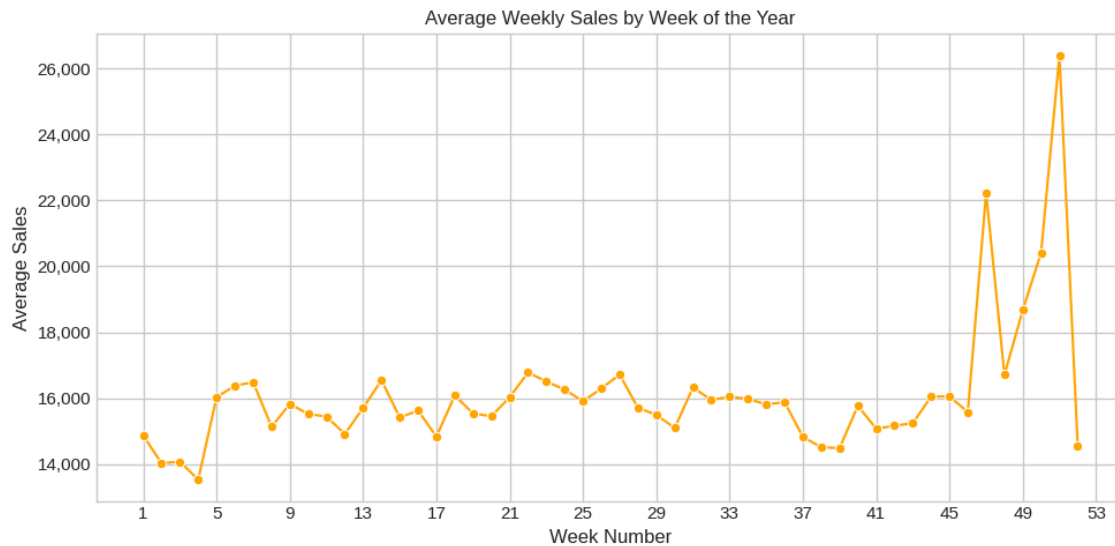


Insight:

A small subset of stores consistently outperforms others in terms of average weekly sales, which may relate to geographic location, size, or customer base.

4.7 Average Sales by Week of the Year

This seasonal analysis shows how sales vary by week number across the year.



Insight:

Sales increase sharply around certain weeks, notably in late November and December, confirming strong seasonal demand patterns that should be captured by the model.

5. BASELINE MODEL (LINEAR REGRESSION)

To establish a performance benchmark, we developed a baseline prediction model using **Linear Regression**. The model was trained on the cleaned and feature-engineered dataset, incorporating variables such as store size, store type, total markdowns, economic indicators (e.g., CPI, unemployment), and time-based features (month and week number).

The data was split into training (80%) and testing (20%) sets without shuffling, to preserve the temporal sequence of the sales data, which is critical in time series forecasting.

The model was implemented using the scikit-learn library, and evaluated on the test set using three standard regression metrics:

- **Mean Absolute Error (MAE):** 11,896.82
- **Root Mean Squared Error (RMSE):** 18,534.27
- **R-squared (R^2):** 0.0515

The **MAE** indicates that, on average, the model's predictions differ from actual weekly sales by approximately \$11,896, which is a considerable deviation given typical sales volumes.

The **RMSE**, at over \$18,500, confirms that the model produces some notably large prediction errors, as this metric penalizes large deviations more heavily than MAE.

The **R^2 value** of 0.0515 suggests that the model explains less than 6% of the variance in the target variable. This means the linear model performs only marginally better than using the overall mean as a prediction for every case.

6. ADVANCED MODELS (RANDOM FOREST, GRADIENT BOOSTING)

To improve prediction accuracy beyond the linear baseline, we implemented two advanced ensemble methods: **Random Forest Regressor** and **Gradient Boosting Regressor**. Both models showed substantial performance improvements, capturing complex, non-linear relationships within the data.

Random Forest Regressor

- **Mean Absolute Error (MAE):** 4,699.33
- **Root Mean Squared Error (RMSE):** 9,639.78
- **R-squared (R^2):** 0.7434

The Random Forest model reduced the average prediction error by more than 60% compared to the baseline and explained approximately 74.3% of the variance in weekly sales. These results indicate a strong capacity to model intricate dependencies and feature interactions within the dataset.

Gradient Boosting Regressor

- **Mean Absolute Error (MAE):** 6,775.90
- **Root Mean Squared Error (RMSE):** 10,723.24
- **R-squared (R^2):** 0.6825

The Gradient Boosting model also significantly outperformed the baseline, explaining about 68.3% of the variance. Although its error metrics are slightly higher than those of Random Forest, it still demonstrates robust predictive performance, especially in capturing nuanced patterns over time.

7. MODEL COMPARISON AND EVALUATION

Both ensemble models clearly outperform the baseline Linear Regression, reinforcing the need for non-linear, tree-based algorithms in forecasting weekly retail sales. They are better suited to handle the interactions between economic indicators, markdowns, seasonal factors, and store characteristics.

Among the two, the Random Forest Regressor achieved the lowest error rates and highest variance explanation, making it the strongest overall performer. It provides a solid foundation for further refinement, hyperparameter tuning, and potential integration into a production-level forecasting system.

Appendix 1: Feature Dictionary - walmart_preprocessed.csv

Feature	Type	Description
Store	Integer	Unique ID of the store.
Dept	Integer	Department number within the store.
Size	Numeric	Physical size of the store in square feet.
Temperature	Numeric	Average temperature for the week (normalized between 0 and 1).
Fuel_Price	Numeric	Average fuel price in the area (normalized).
CPI	Numeric	Consumer Price Index – economic indicator (normalized).
Unemployment	Numeric	Unemployment rate in the area (normalized).
Total_MarkDown	Numeric	Sum of all promotional markdowns (MarkDown1 to MarkDown5).
Month	Integer	Month extracted from the date (1–12).
Week	Integer	ISO week number of the year extracted from the date.
DayOfWeek	Integer (0–6)	Day of the week (0 = Monday, 6 = Sunday).
IsWeekend	Binary (0 or 1)	1 if the day falls on a weekend (Saturday/Sunday), 0 otherwise.
Type_B	Binary (0 or 1)	Dummy variable indicating if the store is of Type B (Type A = reference).
Type_C	Binary (0 or 1)	Dummy variable indicating if the store is of Type C.
IsHoliday_True	Binary (0 or 1)	1 if the week contains a public holiday, 0 otherwise.
Weekly_Sales	Numeric (Target)	Weekly sales for a specific department and store (target variable).